

Dynamic Programming: DP on Strings

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: DP on Strings	1
1.1	One Table, Two Pointers, Two Strings	1
1.2	Longest Common Subsequence	2
1.3	Print the Longest Common Subsequence	5
1.4	Longest Common Substring	7
1.5	Shortest Common Supersequence	10
1.6	Distinct Subsequences	12
1.7	Edit Distance	15
1.8	Delete Operations for Two Strings	19
1.9	Longest Palindromic Subsequence	20
1.10	Minimum Insertions to Make a String Palindrome	21
1.11	Wildcard Matching	23

Chapter 1

Dynamic Programming: DP on Strings

Two strings rarely need a third dimension. Almost everything in this chapter is answered by a single 2D table, indexed by one pointer into each string, where the recurrence is just: do the characters at these two pointers match, and if so, how does that agreement combine with the answer for both pointers advanced together?

1.1 One Table, Two Pointers, Two Strings

Every problem in this chapter compares two strings s_1 (length n) and s_2 (length m) using a DP state (i, j) , where i indexes a prefix (or suffix) of s_1 and j indexes a prefix (or suffix) of s_2 . The recurrence almost always branches on a single question: does $s_1[i] = s_2[j]$?

The Core Recurrence Almost Every Problem Specializes

- **If $s_1[i] = s_2[j]$:** this matched pair of characters is worth keeping, so add 1 (or otherwise credit the match) and recurse on *both* pointers advanced: $f(i, j) = 1 + f(i-1, j-1)$ (for subsequence-length problems), or a similarly structured combination for other problems.
- **If $s_1[i] \neq s_2[j]$:** the characters cannot both be used in this match, so the best answer comes from advancing *one* pointer at a time and taking whichever is better: $f(i, j) = \max(f(i-1, j), f(i, j-1))$, or, for edit-distance-style problems, the minimum over several specific single-step or dual-step operations (insert, delete, replace).

The entire chapter is variations on this one template: **Longest Common Subsequence** establishes it in its cleanest form; every other section either reuses it directly (Shortest Common Supersequence, Distinct Subsequences), reduces to it (Longest Palindromic Subsequence, Delete Operations, Minimum Insertions for Palindrome), or extends it with a small number of additional operations (Edit Distance, Wildcard Matching).

Why Recursing on a Suffix-or-Prefix Pair, Not a Single Index, Is the New Idea

Every problem in the DP 1D and DP Grids chapters so far has used at most two coupled positions (a row and column on one shared structure, or two agents on the same row). Here, the two indices i and j belong to **independent** strings – $s_1[i]$ and $s_2[j]$ are not from the same array at all, so there is no inherent geometric relationship between them the way there was for grid DP's row/column. The table is still filled exactly like a grid (row by row, or with both loops nested), but the meaning of “the cell at (i, j) ” is now “comparing a prefix of s_1 against a prefix of s_2 ,” a genuinely new state shape worth internalizing carefully in the chapter's first section before relying on it everywhere else.

1.2 Longest Common Subsequence

Problem Statement. Given two strings s_1 (length n) and s_2 (length m), find the length of their **longest common subsequence** (LCS): the longest sequence of characters that appears, in order but not necessarily contiguously, in both strings.

Pattern Recognition

This is the chapter's foundational recurrence, established in its cleanest form: "*longest sequence appearing in order in both strings*" is answered by comparing characters one pair at a time from the end (or start) of each string, branching on whether they match.

Deriving the recurrence

It is conventional in string DP to use a **1-indexed** table, where $f(i, j)$ denotes the answer for the prefixes $s_1[0..i-1]$ (length i) and $s_2[0..j-1]$ (length j) – this lets $f(0, j)$ and $f(i, 0)$ cleanly represent "one of the strings is empty," whose LCS length is trivially 0, without needing special-case index arithmetic for negative indices.

$$f(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ 1 + f(i-1, j-1) & \text{if } s_1[i-1] = s_2[j-1], \\ \max(f(i-1, j), f(i, j-1)) & \text{otherwise.} \end{cases}$$

If the last characters of both prefixes match, that character is part of some longest common subsequence, so we credit it and recurse on both prefixes shortened by one. If they don't match, the last character of *at least one* prefix is not part of the LCS (it cannot be part of *both* prefixes' contribution simultaneously, since they disagree), so we try discarding each one in turn and take the better result.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lcsRecursive(int i, int j, string& s1, string& s2) {
5     if (i == 0 || j == 0) return 0;
6
7     if (s1[i - 1] == s2[j - 1]) {
8         return 1 + lcsRecursive(i - 1, j - 1, s1, s2);
9     }
10    return max(lcsRecursive(i - 1, j, s1, s2), lcsRecursive(i, j - 1,
11        s1, s2));
12 }
13
14 int main() {
15     string s1 = "abcde", s2 = "ace";
16     cout << lcsRecursive(s1.size(), s2.size(), s1, s2) << endl; // 3
17     return 0;
18 }
```

Approach 1 Complexity

Time. $O(2^{n+m})$ – two branches per mismatch, depth up to $n + m$. **Space.** $O(n + m)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lcsMemo(int i, int j, string& s1, string& s2, vector<vector<int>>&
memo) {
5     if (i == 0 || j == 0) return 0;
6     if (memo[i][j] != -1) return memo[i][j];
7
8     if (s1[i - 1] == s2[j - 1]) {
9         return memo[i][j] = 1 + lcsMemo(i - 1, j - 1, s1, s2, memo);
10    }
11    return memo[i][j] = max(lcsMemo(i - 1, j, s1, s2, memo),
12                           lcsMemo(i, j - 1, s1, s2, memo));
13 }
14
15 int main() {
16     string s1 = "abcde", s2 = "ace";
17     int n = s1.size(), m = s2.size();
18     vector<vector<int>> memo(n + 1, vector<int>(m + 1, -1));
19     cout << lcsMemo(n, m, s1, s2, memo) << endl; // 3
20     return 0;
21 }

```

Approach 2 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ memo + $O(n + m)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lcsTabulation(string& s1, string& s2) {
5     int n = s1.size(), m = s2.size();
6     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
7
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= m; j++) {
10             if (s1[i - 1] == s2[j - 1]) {
11                 dp[i][j] = 1 + dp[i - 1][j - 1];
12             } else {
13                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
14             }
15         }
16     }
17     return dp[n][m];
18 }
19
20 int main() {
21     string s1 = "abcde", s2 = "ace";
22     cout << lcsTabulation(s1, s2) << endl; // 3
23     return 0;
24 }

```

Dry run of the tabulation table

Dry Run. Take $s_1 = \text{"abcde"}$, $s_2 = \text{"ace"}$.

	""	a	c	e
""	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

At (i, j) corresponding to c (row) and c (column): match, so $dp = 1 + dp[\text{prev row}][\text{prev col}] = 1 + 1 = 2$. At the final cell (e, e): match, $dp = 1 + 2 = 3$. The LCS length is 3 (the subsequence "ace" itself).

Approach 3 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lcsSpaceOptimized(string& s1, string& s2) {
5     int n = s1.size(), m = s2.size();
6     vector<int> prev(m + 1, 0), current(m + 1, 0);
7
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= m; j++) {
10             if (s1[i - 1] == s2[j - 1]) {
11                 current[j] = 1 + prev[j - 1];
12             } else {
13                 current[j] = max(prev[j], current[j - 1]);
14             }
15         }
16         prev = current;
17     }
18     return prev[m];
19 }
20
21 int main() {
22     string s1 = "abcde", s2 = "ace";
23     cout << lcsSpaceOptimized(s1, s2) << endl; // 3
24     return 0;
25 }
```

Approach 4 Complexity

Time. $O(n \cdot m)$. **Space.** $O(m)$ for two rolling rows.

Key Takeaway

This match/mismatch recurrence – credit a match and advance both pointers, or try discarding one character from each side on a mismatch – is the single template the rest of

this chapter builds from. Memorize its shape (not just its code) before moving on, since every later section either reuses it verbatim on a transformed input (reversing one string, concatenating two strings) or extends its mismatch case with extra operations.

1.3 Print the Longest Common Subsequence

Problem Statement. Given two strings s_1 and s_2 , construct (not just measure the length of) one valid longest common subsequence.

Pattern Recognition

“*Print/construct the actual solution, not just its length/count*” is a new keyword pattern this chapter introduces: once a tabulation table is built, **walking it backward** from the final answer cell, re-deriving at each step which recurrence case produced that cell’s value, reconstructs an actual optimal solution – without needing any new DP derivation, only a careful reverse walk through the already-computed table.

Understanding the reconstruction

The Section 1.2 tabulation table $dp[i][j]$ must be built in full first (this requires the complete $O(nm)$ table, not the space-optimized $O(m)$ rolling-row version, since reconstruction needs to look back arbitrarily far). Starting at $dp[n][m]$, walk backward: if $s_1[i-1] = s_2[j-1]$, this character is part of the LCS – record it and move diagonally to $(i-1, j-1)$. Otherwise, move to whichever of $(i-1, j)$ or $(i, j-1)$ has the larger dp value (matching whichever branch the forward recurrence actually took to produce $dp[i][j]$ ’s value). Continue until reaching a row or column of 0. The characters recorded, read in reverse order (since we walked backward from the end), form one valid LCS.

Why retracing the recurrence’s decision backward is correct

The tabulation table already encodes, implicitly, which choice (match, or which discard) was responsible for each cell’s value – we simply need to recompute that same comparison once more during the backward walk, since the table only stores the resulting *length*, not which branch produced it. Because the forward recurrence’s correctness was already established in Section 1.2, redoing its exact decision rule in reverse is guaranteed to retrace a genuine optimal path through the table, character by character, until the base case (an empty prefix) is reached.

Algorithm

1. Build the full dp table as in Section 1.2’s tabulation.
2. $i \leftarrow n, j \leftarrow m, result \leftarrow ""$.
3. While $i > 0$ and $j > 0$:
 - (a) If $s_1[i-1] = s_2[j-1]$: prepend $s_1[i-1]$ to $result$; $i \leftarrow i-1, j \leftarrow j-1$.
 - (b) Else if $dp[i-1][j] \geq dp[i][j-1]$: $i \leftarrow i-1$.
 - (c) Else: $j \leftarrow j-1$.
4. Return $result$.

Dry run

Dry Run. Using Section 1.2's table for $s_1 = \text{"abcde"}$, $s_2 = \text{"ace"}$.

- Start at $(i, j) = (5, 3)$: $s_1[4] = \text{'e'}$, $s_2[2] = \text{'e'}$, match. Record 'e'. Move to $(4, 2)$.
- $(4, 2)$: $s_1[3] = \text{'d'}$, $s_2[1] = \text{'c'}$, no match. $dp[3][2] = 2 \geq dp[4][1] = 1$: move to $(3, 2)$.
- $(3, 2)$: $s_1[2] = \text{'c'}$, $s_2[1] = \text{'c'}$, match. Record 'c'. Move to $(2, 1)$.
- $(2, 1)$: $s_1[1] = \text{'b'}$, $s_2[0] = \text{'a'}$, no match. $dp[1][1] = 1 \geq dp[2][0] = 0$: move to $(1, 1)$.
- $(1, 1)$: $s_1[0] = \text{'a'}$, $s_2[0] = \text{'a'}$, match. Record 'a'. Move to $(0, 0)$. Loop ends ($i = 0$).

Characters recorded, in the order found (backward): e, c, a. Reversing gives "ace" – the LCS itself.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string printLCS(string& s1, string& s2) {
5     int n = s1.size(), m = s2.size();
6     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
7
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= m; j++) {
10             if (s1[i - 1] == s2[j - 1]) {
11                 dp[i][j] = 1 + dp[i - 1][j - 1];
12             } else {
13                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
14             }
15         }
16     }
17
18     string result;
19     int i = n, j = m;
20     while (i > 0 && j > 0) {
21         if (s1[i - 1] == s2[j - 1]) {
22             result.push_back(s1[i - 1]);
23             i--; j--;
24         } else if (dp[i - 1][j] >= dp[i][j - 1]) {
25             i--;
26         } else {
27             j--;
28         }
29     }
30     reverse(result.begin(), result.end());
31     return result;
32 }
33
34 int main() {
35     string s1 = "abcde", s2 = "ace";
36     cout << printLCS(s1, s2) << endl; // ace
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(n \cdot m)$ to build the table, plus $O(n + m)$ for the backward walk – dominated by the table construction. **Space Complexity.** $O(n \cdot m)$ for the full table – **not** reducible to a rolling row, since reconstruction needs access to arbitrary earlier rows during the backward walk.

Key Takeaway

“Print/construct the solution” problems always need the *full* tabulation table kept in memory, sacrificing the space optimization that a length/count-only version of the same problem could use – the backward walk that reconstructs an answer needs to revisit cells the rolling-row trick would have already discarded. This trade-off (full table for reconstruction vs. rolling row for the value alone) is worth remembering whenever a problem explicitly asks for the solution itself.

1.4 Longest Common Substring

Problem Statement. Given two strings s_1 and s_2 , find the length of their longest common **substring**: a longest run of **contiguous** characters that appears, unbroken, in both strings.

Pattern Recognition

The keyword shift from *subsequence* (Section 1.2) to *substring* – contiguity now required – breaks the mismatch case of the LCS recurrence entirely: a substring cannot “skip” a mismatched character and continue, so a mismatch must **reset** the current run to length 0, rather than falling back to the best of two neighboring sub-answers.

Deriving the recurrence

Let $f(i, j)$ be the length of the common substring **ending exactly at** $s_1[i - 1]$ and $s_2[j - 1]$ (not the best answer over all prefixes, as in LCS – specifically the run ending right here):

$$f(i, j) = \begin{cases} 1 + f(i-1, j-1) & \text{if } s_1[i-1] = s_2[j-1], \\ 0 & \text{otherwise.} \end{cases}$$

Because the answer is not necessarily at $f(n, m)$ (the longest common run could end anywhere in either string), we additionally track a running global maximum over every $f(i, j)$ computed.

Approach 1: Recursion

Why a Direct Recursive Translation Is Awkward Here

Unlike every previous problem in this book, $f(i, j)$ alone does not directly answer the original question – the answer is $\max_{i,j} f(i, j)$, a quantity that is naturally accumulated during a forward (tabulation-style) sweep, not easily expressed as a single top-down recursive call. For this reason, Longest Common Substring is typically presented starting from tabulation, with the recursive/memoized forms understood as computing the same per-cell $f(i, j)$ values, just collected via top-down calls instead of a bottom-up loop.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int f(int i, int j, string& s1, string& s2, vector<vector<int>>& memo,
5     int& best) {
6     if (i == 0 || j == 0) return 0;
7     if (memo[i][j] != -1) return memo[i][j];
8
9     int result = 0;
10    if (s1[i - 1] == s2[j - 1]) {
11        result = 1 + f(i - 1, j - 1, s1, s2, memo, best);
12    }
13    best = max(best, result);
14    return memo[i][j] = result;
15 }
16
17 int longestCommonSubstringMemo(string& s1, string& s2) {
18     int n = s1.size(), m = s2.size();
19     vector<vector<int>> memo(n + 1, vector<int>(m + 1, -1));
20     int best = 0;
21
22     for (int i = 1; i <= n; i++) {
23         for (int j = 1; j <= m; j++) {
24             f(i, j, s1, s2, memo, best);
25         }
26     }
27     return best;
28 }
29
30 int main() {
31     string s1 = "abcde", s2 = "abfce";
32     cout << longestCommonSubstringMemo(s1, s2) << endl; // 2
33     return 0;
34 }

```

Approach 2 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ memo.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int longestCommonSubstringTabulation(string& s1, string& s2) {
5     int n = s1.size(), m = s2.size();
6     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
7     int best = 0;
8
9     for (int i = 1; i <= n; i++) {
10        for (int j = 1; j <= m; j++) {
11            if (s1[i - 1] == s2[j - 1]) {
12                dp[i][j] = 1 + dp[i - 1][j - 1];
13            } else {

```

```

14         dp[i][j] = 0;
15     }
16     best = max(best, dp[i][j]);
17 }
18 }
19 return best;
20 }
21
22 int main() {
23     string s1 = "abcde", s2 = "abfce";
24     cout << longestCommonSubstringTabulation(s1, s2) << endl; // 2
25     return 0;
26 }

```

Dry run of the tabulation table

Dry Run. Take $s_1 = \text{"abcde"}$, $s_2 = \text{"abfce"}$.

	" "	a	b	f	c	e
" "	0	0	0	0	0	0
a	0	1	0	0	0	0
b	0	0	2	0	0	0
c	0	0	0	0	1	0
d	0	0	0	0	0	0
e	0	0	0	0	0	1

The running maximum reaches 2 at cell (b, b) – the common substring "ab" – and never exceeds it elsewhere (every other match is isolated, contributing only length 1 runs).

Approach 3 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ table, no recursion stack.

Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int longestCommonSubstringSpaceOptimized(string& s1, string& s2) {
5      int n = s1.size(), m = s2.size();
6      vector<int> prev(m + 1, 0), current(m + 1, 0);
7      int best = 0;
8
9      for (int i = 1; i <= n; i++) {
10         for (int j = 1; j <= m; j++) {
11             if (s1[i - 1] == s2[j - 1]) {
12                 current[j] = 1 + prev[j - 1];
13             } else {
14                 current[j] = 0;
15             }
16             best = max(best, current[j]);
17         }
18         prev = current;
19     }
20     return best;

```

```

21 }
22
23 int main() {
24     string s1 = "abcde", s2 = "abfce";
25     cout << longestCommonSubstringSpaceOptimized(s1, s2) << endl; // 2
26     return 0;
27 }

```

Approach 4 Complexity

Time. $O(n \cdot m)$. **Space.** $O(m)$ for two rolling rows.

Key Takeaway

“Subsequence” (gaps allowed) and “substring” (no gaps allowed) are the single most important keyword distinction in string DP, and they produce genuinely different recurrences – LCS falls back to a neighboring best-of-two on a mismatch, while Longest Common Substring resets to zero, because a broken run cannot be patched back together. Always check this one word before assuming a string problem is “LCS in disguise.”

1.5 Shortest Common Supersequence

Problem Statement. Given two strings s_1 (length n) and s_2 (length m), construct the **shortest** string that has **both** s_1 and s_2 as subsequences.

Pattern Recognition

“Shortest string containing both as subsequences” is the “opposite” framing of Section 1.2’s LCS, and the two are linked by a clean formula: every character the two strings *share* in their LCS needs to appear only **once** in the supersequence (since it serves both strings’ subsequence requirement simultaneously), while every other character of each string must still appear – collapsing the problem into one already-solved LCS computation plus a reconstruction walk almost identical to Section 1.3’s.

Deriving the length formula

If we simply concatenated s_1 and s_2 , the result trivially contains both as subsequences, with length $n + m$ – but every character that is part of *some* common subsequence is being counted **twice** in this concatenation (once from each string), when it only needs to appear once. The longest common subsequence is exactly the largest set of characters we can “merge” this way, so:

$$|SCS| = n + m - |LCS(s_1, s_2)|.$$

Constructing the actual supersequence

Run the same backward walk as Section 1.3, but instead of recording *only* matched characters, record **every** character visited along the way: on a match, record the (shared) character once and move diagonally; on a mismatch, record whichever of $s_1[i - 1]$ or $s_2[j - 1]$ corresponds to the direction moved (following the same $dp[i - 1][j]$ vs. $dp[i][j - 1]$ comparison as before), and move only that one pointer. Once either pointer reaches 0, append the entirety of whatever remains of the other string (every leftover character must still appear, in order, since that string’s subsequence requirement is otherwise unmet).

Algorithm

1. Build the LCS tabulation table for s_1, s_2 (as in Section 1.2).
2. $i \leftarrow n, j \leftarrow m, result \leftarrow ""$.
3. While $i > 0$ and $j > 0$:
 - (a) If $s_1[i - 1] = s_2[j - 1]$: prepend $s_1[i - 1]$; $i--, j--$.
 - (b) Else if $dp[i - 1][j] \geq dp[i][j - 1]$: prepend $s_1[i - 1]$; $i--$.
 - (c) Else: prepend $s_2[j - 1]$; $j--$.
4. Prepend any remaining prefix of s_1 (if $i > 0$) or s_2 (if $j > 0$).
5. Return *result*.

Dry run

Dry Run. Take $s_1 = \text{"abac"}, s_2 = \text{"cab"}$.

LCS of "abac" and "cab" is "ab" (length 2), so $|SCS| = 4 + 3 - 2 = 5$. Constructing: walking backward through the LCS table merges the two strings, interleaving non-matched characters with the two shared a and b, producing one valid result such as "cabac" (length 5) – which indeed contains "abac" as a subsequence (positions 1, 3, 4, 5... more directly, removing the leading c from "cabac" leaves "abac" exactly) and "cab" as a subsequence (the first three characters).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  string shortestCommonSupersequence(string& s1, string& s2) {
5      int n = s1.size(), m = s2.size();
6      vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
7
8      for (int i = 1; i <= n; i++) {
9          for (int j = 1; j <= m; j++) {
10             if (s1[i - 1] == s2[j - 1]) {
11                 dp[i][j] = 1 + dp[i - 1][j - 1];
12             } else {
13                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
14             }
15         }
16     }
17
18     string result;
19     int i = n, j = m;
20     while (i > 0 && j > 0) {
21         if (s1[i - 1] == s2[j - 1]) {
22             result.push_back(s1[i - 1]);
23             i--; j--;
24         } else if (dp[i - 1][j] >= dp[i][j - 1]) {
25             result.push_back(s1[i - 1]);
26             i--;
27         } else {
28             result.push_back(s2[j - 1]);
29             j--;

```

```

30     }
31 }
32 while (i > 0) { result.push_back(s1[i - 1]); i--; }
33 while (j > 0) { result.push_back(s2[j - 1]); j--; }
34
35 reverse(result.begin(), result.end());
36 return result;
37 }
38
39 int main() {
40     string s1 = "abac", s2 = "cab";
41     string scs = shortestCommonSupersequence(s1, s2);
42     cout << scs << " (length " << scs.size() << ")" << endl;
43     return 0;
44 }

```

Complexity Analysis

Time Complexity. $O(n \cdot m)$ to build the LCS table, plus $O(n + m)$ for the reconstruction walk. **Space Complexity.** $O(n \cdot m)$ for the full table (needed for reconstruction, as in Section 1.3).

Key Takeaway

Shortest Common Supersequence is not a new DP recurrence at all – it is LCS’s length formula, $n + m - |LCS|$, plus a reconstruction walk that is a small variation on Print LCS’s (recording every visited character, not only the matches). Whenever a problem can be phrased as “the complement or merge of an already-solved quantity,” check the arithmetic relationship before reaching for a new table.

1.6 Distinct Subsequences

Problem Statement. Given strings s (length n) and t (length m), count the number of distinct ways t can be formed as a subsequence of s (counting different choices of *positions* in s as distinct, even if they spell the same characters).

Pattern Recognition

“Count the number of ways one string appears as a subsequence of another” reuses Section 1.2’s match/mismatch branching, but on a **match**, there are now **two** valid choices to *sum* rather than one branch to take unconditionally: use the current character of s to match t , or skip it and hope a later character of s matches instead – both contribute to the count, since s may contain the needed character multiple times.

Deriving the recurrence

Let $f(i, j)$ be the number of ways $t[0..j - 1]$ (length j) can be formed as a subsequence of $s[0..i - 1]$ (length i).

$$f(i, j) = \begin{cases} f(i-1, j-1) + f(i-1, j) & \text{if } s[i-1] = t[j-1], \\ f(i-1, j) & \text{otherwise.} \end{cases}$$

If the characters match, we may either **use** this occurrence in s to match t ’s current character (advancing both, $f(i-1, j-1)$), or **skip** it, leaving t ’s current character to be matched by

some earlier-yet-unconsidered part of s ($f(i-1, j)$) – both possibilities lead to valid, distinct subsequence selections, so their counts are summed. If the characters don't match, this position in s cannot contribute to matching $t[j-1]$ at all, so only the skip option remains.

Base cases: $f(i, 0) = 1$ for every i (the empty string is a subsequence of anything, in exactly one way – selecting nothing). $f(0, j) = 0$ for every $j > 0$ (a non-empty t cannot be a subsequence of an empty s).

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int distinctSubsequencesRecursive(int i, int j, string& s, string& t) {
5     if (j == 0) return 1;
6     if (i == 0) return 0;
7
8     if (s[i - 1] == t[j - 1]) {
9         return distinctSubsequencesRecursive(i - 1, j - 1, s, t) +
10             distinctSubsequencesRecursive(i - 1, j, s, t);
11     }
12     return distinctSubsequencesRecursive(i - 1, j, s, t);
13 }
14
15 int main() {
16     string s = "babgbag", t = "bag";
17     cout << distinctSubsequencesRecursive(s.size(), t.size(), s, t) <<
18         endl; // 5
19     return 0;
20 }
```

Approach 1 Complexity

Time. $O(2^n)$ in the worst case. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int distinctSubsequencesMemo(int i, int j, string& s, string& t,
5                             vector<vector<long long>>& memo) {
6     if (j == 0) return 1;
7     if (i == 0) return 0;
8     if (memo[i][j] != -1) return memo[i][j];
9
10    long long result;
11    if (s[i - 1] == t[j - 1]) {
12        result = distinctSubsequencesMemo(i - 1, j - 1, s, t, memo) +
13            distinctSubsequencesMemo(i - 1, j, s, t, memo);
14    } else {
15        result = distinctSubsequencesMemo(i - 1, j, s, t, memo);
16    }
17    return memo[i][j] = result;
18 }
19 }
```

```

20 int main() {
21     string s = "babgbag", t = "bag";
22     int n = s.size(), m = t.size();
23     vector<vector<long long>> memo(n + 1, vector<long long>(m + 1, -1))
24     ;
25     cout << distinctSubsequencesMemo(n, m, s, t, memo) << endl; // 5
26     return 0;
27 }

```

Approach 2 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long distinctSubsequencesTabulation(string& s, string& t) {
5      int n = s.size(), m = t.size();
6      vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));
7
8      for (int i = 0; i <= n; i++) dp[i][0] = 1;
9
10     for (int i = 1; i <= n; i++) {
11         for (int j = 1; j <= m; j++) {
12             if (s[i - 1] == t[j - 1]) {
13                 dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
14             } else {
15                 dp[i][j] = dp[i - 1][j];
16             }
17         }
18     }
19     return dp[n][m];
20 }
21
22 int main() {
23     string s = "babgbag", t = "bag";
24     cout << distinctSubsequencesTabulation(s, t) << endl; // 5
25     return 0;
26 }

```

Dry run of the tabulation table

Dry Run. Take $s = \text{"babgbag"}$, $t = \text{"bag"}$.

The five ways "bag" appears as a subsequence of "babgbag" correspond to choosing one of the two 'b's (positions 0, 2, 4... specifically 3 valid 'b' positions combined with valid later 'a', 'g' pairs) in combination with the available 'a' and 'g' occurrences after it; building the table row by row accumulates these combinations automatically via the sum rule, reaching $dp[7][3] = 5$.

Approach 3 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long distinctSubsequencesSpaceOptimized(string& s, string& t) {
5     int n = s.size(), m = t.size();
6     vector<long long> prev(m + 1, 0), current(m + 1, 0);
7     prev[0] = current[0] = 1;
8
9     for (int i = 1; i <= n; i++) {
10         for (int j = 1; j <= m; j++) {
11             if (s[i - 1] == t[j - 1]) {
12                 current[j] = prev[j - 1] + prev[j];
13             } else {
14                 current[j] = prev[j];
15             }
16         }
17         prev = current;
18     }
19     return prev[m];
20 }
21
22 int main() {
23     string s = "babgbag", t = "bag";
24     cout << distinctSubsequencesSpaceOptimized(s, t) << endl; // 5
25     return 0;
26 }

```

Approach 4 Complexity

Time. $O(n \cdot m)$. **Space.** $O(m)$ for two rolling rows.

Key Takeaway

A character match does not always mean “take it unconditionally and advance both pointers” – when the same character could plausibly be matched by *several* different positions in the longer string (as here, where s may contain repeats), a match should branch into **both** “use it” and “save it for later, skip for now,” summing their counts. This is a subtler version of the match case than Section 1.2’s, worth contrasting carefully.

1.7 Edit Distance

Problem Statement. Given two strings s_1 (length n) and s_2 (length m), find the minimum number of operations – **insert**, **delete**, or **replace** a single character – needed to transform s_1 into s_2 .

Pattern Recognition

This is Section 1.2’s match/mismatch template, extended: the **match** case is unchanged (no operation needed, advance both pointers for free), but the **mismatch** case is no longer a simple “discard one side” – it now branches into **three** possible operations, each costing 1 and corresponding to a different way of advancing the two pointers.

Deriving the recurrence

$$f(i, j) = \begin{cases} f(i-1, j-1) & \text{if } s_1[i-1] = s_2[j-1], \\ 1 + \min(f(i-1, j-1), f(i-1, j), f(i, j-1)) & \text{otherwise.} \end{cases}$$

On a mismatch, three operations are available, each mapping to a specific pointer movement: **replace** $s_1[i-1]$ with $s_2[j-1]$ (both characters consumed, advance both pointers – $f(i-1, j-1)$); **delete** $s_1[i-1]$ (advance only s_1 's pointer, leaving $s_2[j-1]$ still to be matched – $f(i-1, j)$); or **insert** a copy of $s_2[j-1]$ into s_1 (this newly-inserted character matches $s_2[j-1]$ for free, so only s_2 's pointer advances – $f(i, j-1)$). We pay 1 for whichever operation is used, plus the cost of solving the resulting smaller sub-problem, and take the cheapest option.

Base cases: $f(0, j) = j$ (transform an empty s_1 into s_2 's first j characters by j insertions), $f(i, 0) = i$ (transform s_1 's first i characters into an empty string by i deletions).

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int editDistanceRecursive(int i, int j, string& s1, string& s2) {
5     if (i == 0) return j;
6     if (j == 0) return i;
7
8     if (s1[i - 1] == s2[j - 1]) {
9         return editDistanceRecursive(i - 1, j - 1, s1, s2);
10    }
11    int replace = editDistanceRecursive(i - 1, j - 1, s1, s2);
12    int del     = editDistanceRecursive(i - 1, j, s1, s2);
13    int insert  = editDistanceRecursive(i, j - 1, s1, s2);
14    return 1 + min({replace, del, insert});
15 }
16
17 int main() {
18     string s1 = "horse", s2 = "ros";
19     cout << editDistanceRecursive(s1.size(), s2.size(), s1, s2) << endl
20         ; // 3
21     return 0;
22 }
```

Approach 1 Complexity

Time. $O(3^{n+m})$ in the worst case – three branches per mismatch. **Space.** $O(n + m)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int editDistanceMemo(int i, int j, string& s1, string& s2,
5                     vector<vector<int>>& memo) {
6     if (i == 0) return j;
7     if (j == 0) return i;
8     if (memo[i][j] != -1) return memo[i][j];
9 }
```

```

10     if (s1[i - 1] == s2[j - 1]) {
11         return memo[i][j] = editDistanceMemo(i - 1, j - 1, s1, s2, memo);
12     }
13     int replace = editDistanceMemo(i - 1, j - 1, s1, s2, memo);
14     int del      = editDistanceMemo(i - 1, j, s1, s2, memo);
15     int insert   = editDistanceMemo(i, j - 1, s1, s2, memo);
16     return memo[i][j] = 1 + min({replace, del, insert});
17 }
18
19 int main() {
20     string s1 = "horse", s2 = "ros";
21     int n = s1.size(), m = s2.size();
22     vector<vector<int>> memo(n + 1, vector<int>(m + 1, -1));
23     cout << editDistanceMemo(n, m, s1, s2, memo) << endl; // 3
24     return 0;
25 }

```

Approach 2 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ memo + $O(n + m)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int editDistanceTabulation(string& s1, string& s2) {
5      int n = s1.size(), m = s2.size();
6      vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
7
8      for (int j = 0; j <= m; j++) dp[0][j] = j;
9      for (int i = 0; i <= n; i++) dp[i][0] = i;
10
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= m; j++) {
13             if (s1[i - 1] == s2[j - 1]) {
14                 dp[i][j] = dp[i - 1][j - 1];
15             } else {
16                 dp[i][j] = 1 + min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]});
17             }
18         }
19     }
20     return dp[n][m];
21 }
22
23 int main() {
24     string s1 = "horse", s2 = "ros";
25     cout << editDistanceTabulation(s1, s2) << endl; // 3
26     return 0;
27 }

```

Dry run of the tabulation table

Dry Run. Take $s_1 = \text{"horse"}$, $s_2 = \text{"ros"}$.

	"	r	o	s
"	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

The answer is $dp[5][3] = 3$: one optimal sequence is "horse" $\rightarrow$ "rorse" (replace h $\rightarrow$ r) $\rightarrow$ "rose" (delete the second r) $\rightarrow$ "ros" (delete e): 3 operations.

Approach 3 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int editDistanceSpaceOptimized(string& s1, string& s2) {
5     int n = s1.size(), m = s2.size();
6     vector<int> prev(m + 1), current(m + 1);
7     for (int j = 0; j <= m; j++) prev[j] = j;
8
9     for (int i = 1; i <= n; i++) {
10        current[0] = i;
11        for (int j = 1; j <= m; j++) {
12            if (s1[i - 1] == s2[j - 1]) {
13                current[j] = prev[j - 1];
14            } else {
15                current[j] = 1 + min({prev[j - 1], prev[j], current[j - 1]});
16            }
17        }
18        prev = current;
19    }
20    return prev[m];
21 }
22
23 int main() {
24     string s1 = "horse", s2 = "ros";
25     cout << editDistanceSpaceOptimized(s1, s2) << endl; // 3
26     return 0;
27 }

```

Approach 4 Complexity

Time. $O(n \cdot m)$. **Space.** $O(m)$ for two rolling rows.

Key Takeaway

Edit Distance generalizes the LCS template's mismatch case from "pick the better of two discards" to "pick the cheapest of three specific operations," each mapping to a distinct, intuitive pointer movement. Once this three-way branch is understood, Delete Operations (Section 1.8) is recoverable as the special case where only the delete operation is permitted.

1.8 Delete Operations for Two Strings

Problem Statement. Given two strings s_1 (length n) and s_2 (length m), find the minimum number of character **deletions** (from either string) needed to make the two strings **equal**.

Pattern Recognition

This is Section 1.7's Edit Distance restricted to **only the delete operation** – and that restriction collapses it into a direct reduction to Section 1.2's LCS, rather than needing its own three-way recurrence at all.

Understanding the reduction

If the only allowed operation is deletion, the two strings can only ever become equal by each reducing down to some common subsequence of both – deleting everything that isn't part of that shared subsequence. To minimize total deletions, we want to keep as much as possible, which means keeping the **longest** common subsequence and deleting everything else:

$$\text{minDeletions} = (n - |LCS(s_1, s_2)|) + (m - |LCS(s_1, s_2)|).$$

The first term is how many characters of s_1 are deleted (everything not part of the kept LCS); the second is the same for s_2 .

All Four Approaches

Every stage reuses Section 1.2's LCS implementation unchanged; only this final arithmetic combination is new.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int lcsSpaceOptimized(string& s1, string& s2) {
5      int n = s1.size(), m = s2.size();
6      vector<int> prev(m + 1, 0), current(m + 1, 0);
7
8      for (int i = 1; i <= n; i++) {
9          for (int j = 1; j <= m; j++) {
10             if (s1[i - 1] == s2[j - 1]) {
11                 current[j] = 1 + prev[j - 1];
12             } else {
13                 current[j] = max(prev[j], current[j - 1]);
14             }
15         }
16         prev = current;
17     }
18     return prev[m];
19 }
20
21 int minDeletions(string& s1, string& s2) {
22     int lcsLen = lcsSpaceOptimized(s1, s2);
23     return (s1.size() - lcsLen) + (s2.size() - lcsLen);
24 }
25
26 int main() {
27     string s1 = "sea", s2 = "eat";
28     cout << minDeletions(s1, s2) << endl; // 2
29     return 0;

```

30 }

Dry run

Dry Run. Take $s_1 = \text{"sea"}$, $s_2 = \text{"eat"}$.

LCS of "sea" and "eat" is "ea" (length 2). $\text{minDeletions} = (3 - 2) + (3 - 2) = 1 + 1 = 2$: delete 's' from s_1 (leaving "ea") and delete 't' from s_2 (leaving "ea") – both strings become "ea" with 2 total deletions.

Complexity Analysis

Time Complexity. $O(n \cdot m)$, inherited directly from LCS. **Space Complexity.** $O(m)$ for the rolling-row LCS.

Key Takeaway

Whenever an edit-distance-style problem restricts the allowed operations to a strict subset (here, deletions only), check whether the restricted version reduces to a simpler, already-solved problem rather than re-deriving a new three-way (or however-many-way) recurrence. “Delete only, from both strings, to reach equality” is exactly “maximize the shared kept subsequence,” i.e. LCS, in disguise.

1.9 Longest Palindromic Subsequence

Problem Statement. Given a string s , find the length of its **longest palindromic subsequence** (a subsequence that reads the same forward and backward).

Pattern Recognition

“Longest subsequence that is a palindrome” reduces directly to Section 1.2: a palindrome reads identically forward and backward, so a subsequence of s is a palindrome exactly when it is *also* a subsequence of s ’s **reverse** – making the longest palindromic subsequence of s exactly the longest common subsequence of s and $\text{reverse}(s)$.

Understanding the reduction

Let $r = \text{reverse}(s)$. Any common subsequence of s and r is, by construction, a sequence of characters appearing in order in s that *also* appears in order in r – but appearing in order in r is the same as appearing in *reverse* order in s . A sequence that is both a subsequence of s in forward order and a subsequence of s in reverse order is precisely a palindromic subsequence of s . So:

$$|LPS(s)| = |LCS(s, \text{reverse}(s))|.$$

All Four Approaches

Every stage is Section 1.2’s LCS implementation, called with $s_1 = s$ and $s_2 = \text{reverse}(s)$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lcsSpaceOptimized(string& s1, string& s2) {
5     int n = s1.size(), m = s2.size();
6     vector<int> prev(m + 1, 0), current(m + 1, 0);
7

```

```

8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= m; j++) {
10            if (s1[i - 1] == s2[j - 1]) {
11                current[j] = 1 + prev[j - 1];
12            } else {
13                current[j] = max(prev[j], current[j - 1]);
14            }
15        }
16        prev = current;
17    }
18    return prev[m];
19 }
20
21 int longestPalindromicSubsequence(string s) {
22     string r = s;
23     reverse(r.begin(), r.end());
24     return lcsSpaceOptimized(s, r);
25 }
26
27 int main() {
28     cout << longestPalindromicSubsequence("bbbab") << endl; // 4
29     return 0;
30 }

```

Dry run

Dry Run. Take $s = \text{"bbbab"}$.

$r = \text{"babbb"}$. The LCS of "bbbab" and "babbb" is "bbbb" (length 4), which is indeed a palindrome and a subsequence of the original "bbbab" (drop the 'a').

Complexity Analysis

Time Complexity. $O(n^2)$ (since $|s_1| = |s_2| = n$), inherited directly from LCS. **Space Complexity.** $O(n)$ for the rolling-row LCS.

Key Takeaway

“Longest palindromic X” problems frequently reduce to “longest common X between the string and its reverse” – a reduction worth checking first before writing a dedicated palindrome recurrence (which does also exist for this specific problem, expanding outward from the center, but the LCS reduction shown here requires no new derivation at all).

1.10 Minimum Insertions to Make a String Palindrome

Problem Statement. Given a string s of length n , find the minimum number of characters that must be **inserted** (anywhere in the string) to make it a palindrome.

Pattern Recognition

A second reduction layered directly on top of Section 1.9: the characters *not* part of the longest palindromic subsequence are exactly the ones that need a “partner” inserted to balance them into a palindrome, so the answer is simply n minus the LPS length.

Understanding the reduction

Any palindromic subsequence of s , of length k , can be extended to a full palindrome covering all of s by inserting, for every character of s *not* part of that subsequence, a matching mirrored character on the opposite side. The longest palindromic subsequence minimizes the number of characters left over needing such a partner, so:

$$\text{minInsertions} = n - |LPS(s)|.$$

All Four Approaches

Every stage reuses Section 1.9's implementation (itself a reduction to LCS) unchanged.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int lcsSpaceOptimized(string& s1, string& s2) {
5      int n = s1.size(), m = s2.size();
6      vector<int> prev(m + 1, 0), current(m + 1, 0);
7
8      for (int i = 1; i <= n; i++) {
9          for (int j = 1; j <= m; j++) {
10             if (s1[i - 1] == s2[j - 1]) {
11                 current[j] = 1 + prev[j - 1];
12             } else {
13                 current[j] = max(prev[j], current[j - 1]);
14             }
15         }
16         prev = current;
17     }
18     return prev[m];
19 }
20
21 int minInsertionsForPalindrome(string s) {
22     string r = s;
23     reverse(r.begin(), r.end());
24     int lpsLen = lcsSpaceOptimized(s, r);
25     return (int)s.size() - lpsLen;
26 }
27
28 int main() {
29     cout << minInsertionsForPalindrome("zzazz") << endl;    // 0 (already
30         a palindrome)
31     cout << minInsertionsForPalindrome("mbadm") << endl;    // 2
32     return 0;
33 }
```

Dry run

Dry Run. Take $s = \text{"mbadm"}$ ($n = 5$).

The longest palindromic subsequence of "mbadm" is "mam" (length 3): the characters m, a, m appear in this order at indices 0, 2, 4, and no length-4 palindromic subsequence exists (no other character repeats). $\text{minInsertions} = 5 - 3 = 2$.

Complexity Analysis

Time Complexity. $O(n^2)$, inherited from the LCS-based LPS computation. **Space Complexity.** $O(n)$ for the rolling-row LCS.

Key Takeaway

This problem closes a three-link reduction chain that is worth seeing as a whole: Minimum Insertions for Palindrome reduces to Longest Palindromic Subsequence, which reduces to Longest Common Subsequence. Recognizing chains of reductions like this – rather than treating each problem as requiring its own fresh derivation – is the single most efficient way to work through a sheet of dozens of nominally distinct DP problems.

1.11 Wildcard Matching

Problem Statement. Given a string s and a pattern p containing literal characters, '?' (matches any **single** character), and '*' (matches any sequence of characters, **including the empty sequence**), determine whether p matches s **entirely**.

Pattern Recognition

This closing problem of the chapter is a deliberate capstone: it returns to Section 1.2's match/mismatch two-pointer table, but the "match" condition is now governed by **pattern syntax** (literal, ?, or *) rather than simple character equality, and * specifically introduces a branching choice (match zero characters, or match one more) reminiscent of the unbounded knapsack family's "stay at the same index" rule from earlier in this book.

Deriving the recurrence

Let $f(i, j)$ be **true** if $s[0..i-1]$ (length i) matches $p[0..j-1]$ (length j). Branch on $p[j-1]$:

- If $p[j-1]$ is a literal character: it must equal $s[i-1]$ exactly (so $i > 0$ is required), and the rest must match: $f(i, j) = (i > 0) \wedge (s[i-1] = p[j-1]) \wedge f(i-1, j-1)$.
- If $p[j-1] = '?'$: it matches any single character of s (so $i > 0$ is required, but no equality check): $f(i, j) = (i > 0) \wedge f(i-1, j-1)$.
- If $p[j-1] = '*'$: it may match **zero** characters of s (so the rest of p must match all of s so far: $f(i, j-1)$), **or** it may match **one more** character of s while remaining available to match further characters too (so $i > 0$ is required: $f(i-1, j)$). Either possibility is acceptable: $f(i, j) = f(i, j-1) \vee ((i > 0) \wedge f(i-1, j))$.

Base cases: $f(0, 0) = \text{true}$ (empty pattern matches empty string). $f(0, j)$ for $j > 0$: **true** only if $p[0..j-1]$ consists *entirely* of '*' characters (each can independently match zero characters, collapsing the whole prefix to nothing). $f(i, 0)$ for $i > 0$: always **false** (a non-empty string cannot be matched by an empty pattern).

Approach 1: Recursion

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool wildcardRecursive(int i, int j, string& s, string& p) {
```

```

5   if (i == 0 && j == 0) return true;
6   if (j == 0) return false; // i > 0 here
7   if (i == 0) {
8       // s is empty: only matches if remaining pattern is all '*'
9       for (int k = 0; k < j; k++) if (p[k] != '*') return false;
10      return true;
11  }
12
13  if (p[j - 1] == '*') {
14      return wildcardRecursive(i, j - 1, s, p) ||
15          wildcardRecursive(i - 1, j, s, p);
16  }
17  if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
18      return wildcardRecursive(i - 1, j - 1, s, p);
19  }
20  return false;
21 }
22
23 int main() {
24     string s = "aa", p = "*";
25     cout << (wildcardRecursive(s.size(), p.size(), s, p) ? "true" : "
26         false") << endl;
27     return 0;
28 }

```

Approach 1 Complexity

Time. $O(2^{n+m})$ in the worst case – two branches per '*' encountered. **Space.** $O(n + m)$ recursion stack.

Approach 2: Memoization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int wildcardMemo(int i, int j, string& s, string& p,
5                  vector<vector<int>>& memo) {
6      if (i == 0 && j == 0) return 1;
7      if (j == 0) return 0;
8      if (i == 0) {
9          for (int k = 0; k < j; k++) if (p[k] != '*') return 0;
10         return 1;
11     }
12     if (memo[i][j] != -1) return memo[i][j];
13
14     bool result;
15     if (p[j - 1] == '*') {
16         result = wildcardMemo(i, j - 1, s, p, memo) ||
17             wildcardMemo(i - 1, j, s, p, memo);
18     } else if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
19         result = wildcardMemo(i - 1, j - 1, s, p, memo);
20     } else {
21         result = false;
22     }
23     return memo[i][j] = result;
24 }
25

```

```

26 int main() {
27     string s = "adceb", p = "*a*b";
28     int n = s.size(), m = p.size();
29     vector<vector<int>> memo(n + 1, vector<int>(m + 1, -1));
30     cout << (wildcardMemo(n, m, s, p, memo) ? "true" : "false") << endl
31         ; // true
32     return 0;
33 }

```

Approach 2 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ memo + $O(n + m)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool wildcardTabulation(string& s, string& p) {
5      int n = s.size(), m = p.size();
6      vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
7      dp[0][0] = true;
8
9      for (int j = 1; j <= m; j++) {
10         bool allStars = true;
11         for (int k = 0; k < j; k++) if (p[k] != '*') { allStars = false; break; }
12         dp[0][j] = allStars;
13     }
14
15     for (int i = 1; i <= n; i++) {
16         for (int j = 1; j <= m; j++) {
17             if (p[j - 1] == '*') {
18                 dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
19             } else if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
20                 dp[i][j] = dp[i - 1][j - 1];
21             } else {
22                 dp[i][j] = false;
23             }
24         }
25     }
26     return dp[n][m];
27 }
28
29 int main() {
30     string s = "adceb", p = "*a*b";
31     cout << (wildcardTabulation(s, p) ? "true" : "false") << endl; //
32         true
33     return 0;
34 }

```

Dry run of the tabulation table

Dry Run. Take $s = \text{"adceb"}$, $p = \text{"*a*b"}$.

Row 0 (s empty): $dp[0][0] = T$; $dp[0][1] = T$ (" $*$ " matches empty); $dp[0][2..4] = F$ (prefix " $*a$ " etc. contain a non- $*$ character, so cannot match the empty string). Filling forward: the

first `*` absorbs "adc", matching the literal `a` against the `a` in "adceb", the second `*` absorbs nothing further, and the literal `b` matches the trailing `b`. The final cell $dp[5][4] = T$, confirming the match.

Approach 3 Complexity

Time. $O(n \cdot m)$. **Space.** $O(n \cdot m)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool wildcardSpaceOptimized(string& s, string& p) {
5     int n = s.size(), m = p.size();
6     vector<bool> prev(m + 1, false), current(m + 1, false);
7     prev[0] = true;
8
9     for (int j = 1; j <= m; j++) {
10         bool allStars = true;
11         for (int k = 0; k < j; k++) if (p[k] != '*') { allStars = false; break; }
12         prev[j] = allStars;
13     }
14
15     for (int i = 1; i <= n; i++) {
16         current[0] = false;
17         for (int j = 1; j <= m; j++) {
18             if (p[j - 1] == '*') {
19                 current[j] = current[j - 1] || prev[j];
20             } else if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
21                 current[j] = prev[j - 1];
22             } else {
23                 current[j] = false;
24             }
25         }
26         prev = current;
27     }
28     return prev[m];
29 }
30
31 int main() {
32     string s = "adceb", p = "*a*b";
33     cout << (wildcardSpaceOptimized(s, p) ? "true" : "false") << endl;
34     // true
35     return 0;
36 }

```

Approach 4 Complexity

Time. $O(n \cdot m)$. **Space.** $O(m)$ for two rolling rows.

Key Takeaway

This chapter closes by combining two of its biggest ideas: the match/mismatch two-pointer table from Section 1.2, and a branching “match zero or match one more” choice for `*` strongly reminiscent of the unbounded-knapsack reuse rule from the DP on Subsequences

chapter. Recognizing that a special pattern character introduces exactly this same kind of “stay and try again” branch is the key insight that unlocks Wildcard Matching (and its sibling, Regular Expression Matching, solved by an almost identical table) without needing an entirely new derivation strategy.